100+ BÀI NÂNG CAO

5 BÍ KÍP CHINH PHỤC PYTHON

LỘ TRÌNH ÔN LUYỆN THI HSG TIN HỌC CẤP TỈNH NĂM 2026 - 2027



" HỌC THÔNG MINH & ÔN LUYỆN & CHẠM TAY VÀO GIẢI THƯỞNG"

BIÊN SOẠN

THẦY TẤN DÂN

Hotline/Zalo: 0937 179 278

THƯ NGỎ TỪ THẦY TẤN DÂN

Kính gửi quý bậc Phụ huynh và các em Học sinh thân mến!

Trong kỷ nguyên số hiện nay, lập trình không còn là một môn học xa lạ, mà đã trở thành "ngôn ngữ của tương lai". Tôi hiểu rằng, đằng sau mỗi quyết định cho con theo học lập trình là biết bao kỳ vọng của quý phụ huynh: kỳ vọng con sẽ rèn luyện được tư duy logic, kỳ vọng con sẽ làm chủ công nghệ thay vì sa đà vào những trò chơi vô bổ, và hơn hết là kỳ vọng con sẽ có một hành trang vững chắc để bước vào những ngôi trường chuyên, lớp chọn hay các trường đại học danh tiếng.

Với nhiều năm kinh nghiệm bồi dưỡng đội tuyển Học sinh giỏi, tôi thấu hiểu những khó khăn mà các em thường gặp phải: từ việc lúng túng trước một bài toán thuật toán khó, đến những lỗi sai "đáng tiếc" khiến các em mất đi cơ hội chạm tay vào giải thưởng.

Cuốn tài liệu **"Bí Kíp Chinh Phục "** này không chỉ là tập hợp những dòng mã, mà là tâm huyết được tôi đúc kết từ hàng nghìn giờ giảng dạy và hàng trăm kỳ thi thực chiến. Cuốn sách này được thiết kế để:

- **Đơn giản hóa** những khái niệm phức tạp thành những bài học dễ hiểu.
- **Trang bị chiến thuật** thực chiến giúp các em tối ưu điểm số trong phòng thi.
- **Khơi dậy niềm đam mê** sáng tạo, giúp các em tự tin chinh phục mọi thử thách.

Tôi tin rằng, mỗi đứa trẻ đều có một tiềm năng vô hạn. Chỉ cần có một lộ trình đúng đắn và một người dẫn đường tận tâm, các em chắc chắn sẽ làm nên những điều kỳ diệu. Giải thưởng Học sinh giỏi cấp Tỉnh không phải là mục đích cuối cùng, mà là bệ phóng để các em tự tin bước ra thế giới.

Cảm ơn quý Phụ huynh đã tin tưởng và đồng hành cùng tôi trên hành trình kiến tạo tương lai cho các em.

Thân ái

Thầy Tấn Dân

MỤC LỤC TỔNG QUÁT:

📂 **PHẦN 1: LÀM CHỦ CÔNG CỤ & KỸ THUẬT PHÒNG THI**

- **Chuyên đề 1:** Cấu trúc chương trình chuẩn để chấm máy tự động (CMS, Themis).
- **Chuyên đề 2:** Kỹ thuật đọc/ghi File (.INP, .OUT) không bao giờ lỗi.
- **Chuyên đề 3:** 10 "Tuyệt chiêu" tối ưu mã nguồn chạy nhanh như C++.
- **Chuyên đề 4:** Cách kiểm soát thời gian và bộ nhớ trong phòng thi.

📂 **PHẦN 2: XỬ LÝ SỐ HỌC & DÃY SỐ (Chiếm 40% số điểm)**

- **Chuyên đề 5:** Số nguyên tố (Sàng Eratosthenes, kiểm tra số nguyên tố cực nhanh).
- **Chuyên đề 6:** Ước chung lớn nhất (GCD), Bội chung nhỏ nhất (LCM) và ứng dụng.
- **Chuyên đề 7:** Phân tích số ra thừa số nguyên tố & các bài toán liên quan.
- **Chuyên đề 8:** Xử lý số siêu lớn (Big Integer) – Thế mạnh tuyệt đối của .

📂 **PHẦN 3: XỬ LÝ XÂU KÝ TỰ & MẢNG (Kỹ năng nền tảng)**

- **Chuyên đề 9:** Kỹ thuật cắt xâu (Slicing) và các hàm xử lý chuỗi "thần thánh".
- **Chuyên đề 10:** Mảng 1 chiều, mảng 2 chiều và kỹ thuật duyệt Ma trận.
- **Chuyên đề 11:** Kỹ thuật Mảng cộng dồn (Prefix Sum) – Giải toán trong O(1).
- **Chuyên đề 12:** Hai con trỏ (Two Pointers) & Cửa sổ trượt (Sliding Window).

📂 **PHẦN 4: THUẬT TOÁN NÂNG CAO (Dành cho giải Nhất, Nhì)**

- **Chuyên đề 13:** Đệ quy & Đệ quy có nhớ (Mồi nhử cho Quy hoạch động).
- **Chuyên đề 14:** Thuật toán Tham lam (Greedy) – Chiến thuật lấy điểm tối đa.
- **Chuyên đề 15:** Chặt nhị phân (Binary Search) trên kết quả.
- **Chuyên đề 16:** Quy hoạch động (Dynamic Programming) – Những bài toán kinh điển (Cái túi, Dãy con tăng dài nhất...).

📂 PHẦN 5: KHO ĐỀ THI THỰC CHIẾN

- **Chuyên đề 17:** Cấu trúc đề thi thực tế.
- **Chuyên đề 18:** Chiến thuật Subtask (Duyệt trâu lấy điểm).
- **Chuyên đề 19:** Các lỗi mất điểm cần tránh.
- **Chuyên đề 20:** Kho 10 đề thi trọng điểm.
- **Tổng hợp:** 20 bộ đề thi HSG cấp tỉnh chính thức của các tỉnh thành (Hà Nội, TP.HCM, Đồng Nai, Tiền Giang...) năm 2024-2025.
- **Đáp án:** Mã nguồn mẫu được tối ưu điểm số kèm giải thích chi tiết.

NỘI DUNG TÓM TẮT

📂 PHẦN 1: CÔNG CỤ & KỸ THUẬT

(Nội dung: Cấu trúc hàm main, sys.stdin, sys.stdout, các mẹo tối ưu tốc độ...)

📂 PHẦN 2: SỐ HỌC (CODE TIẾNG VIỆT)

Chuyên đề 5: Kiểm tra số nguyên tố

```
import math
def kiem_tra_so_nguyen_to(n):
    if n < 2: return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0: return False
    return True
```

(Thầy tiếp tục dán phần nội dung Sàng, UCLN, Phân tích thừa số đã soạn...)

📂 PHẦN 3: XÂU VÀ MẢNG

Chuyên đề 11: Mảng cộng dồn

```
def mang_cong_don(mang_a):
    n = len(mang_a)
    tong_tien_to = [0] * (n + 1)
    for i in range(n):
        tong_tien_to[i+1] = tong_tien_to[i] + mang_a[i]
    return tong_tien_to
```

(Thầy tiếp tục dán phần xử lý xâu, mảng 2 chiều, hai con trỏ...)

📂 PHẦN 4: THUẬT TOÁN NÂNG CAO

Chuyên đề 15: Chặt nhị phân

```
def tim_kiem_nhi_phan(mang, x):
    trai, phai = 0, len(mang) - 1
    while trai <= phai:
```

```
        giua = (trai + phai) // 2
        if mang[giua] == x: return giua
        elif mang[giua] < x: trai = giua + 1
        else: phai = giua - 1
    return -1
```

📂 PHẦN 5: CHIẾN THUẬT PHÒNG THI

(Nội dung về cách phân bổ thời gian, cách "duyệt trâu" ăn điểm lẻ và 10 bộ đề thi...)

NỘI DUNG CHI TIẾT:

📂 PHẦN 1: LÀM CHỦ CÔNG CỤ & KỸ THUẬT PHÒNG THI

Phần này là "nền móng" giúp các em tránh những lỗi mất điểm đáng tiếc và tối ưu hóa hiệu suất làm bài trước khi bắt đầu giải các thuật toán khó.

Chuyên đề 1: Cấu trúc chương trình chuẩn để chấm máy tự động (CMS, Themis)

Để các hệ thống chấm bài tự động như **Themis** hay **CMS** hoạt động chính xác, mã nguồn cần được tổ chức gọn gàng và tránh các lỗi thực thi thừa.

- **Sử dụng hàm main():** Luôn đưa logic chính vào một hàm để dễ quản lý và tăng tốc độ xử lý của .
- **Cấu trúc chuẩn:**

```
import sys

def solve():
    # Viết nội dung giải thuật tại đây
    pass

if __name__ == "__main__":
    # Cài đặt đọc ghi file (nếu có)
    # sys.stdin = open('TENBAI.INP', 'r')
    # sys.stdout = open('TENBAI.OUT', 'w')
    solve()
```

- **Lưu ý:** Tuyệt đối không sử dụng các lệnh chờ như input() hoặc os.system("pause") ở cuối bài vì máy chấm sẽ bị treo và tính lỗi **Time Limit Exceeded (TLE)**.

Chuyên đề 2: Kỹ thuật đọc/ghi File (.INP, .OUT) không bao giờ lỗi

Trong các kỳ thi HSG, việc sai tên File hoặc sai định dạng đọc file là lỗi phổ biến nhất khiến học sinh bị 0 điểm cả bài.

- Cách 1: Chuyển hướng Standard Input/Output (Khuyên dùng)

Cách này giúp em giữ nguyên code input() và print(), chỉ cần thêm 2 dòng ở đầu bài.

```
import sys
sys.stdin = open('DULIEU.INP', 'r')
sys.stdout = open('DULIEU.OUT', 'w')

# Sau đó dùng input() và print() như bình thường
n = int(input())
print(n)
```

- Cách 2: Sử dụng Context Manager (An toàn nhất)

Đảm bảo file luôn được đóng sau khi đọc/ghi, tránh mất dữ liệu.

```
with open('DULIEU.INP', 'r') as f_in:
    data = f_in.read().split()

with open('DULIEU.OUT', 'w') as f_out:
    f_out.write(str(result))
```

- **Mẹo nhỏ:** Luôn kiểm tra kỹ tên file (chữ hoa, chữ thường) theo đúng yêu cầu của đề bài.

Chuyên đề 3: 10 "Tuyệt chiêu" tối ưu mã nguồn chạy nhanh như C++

 thường chậm hơn C++, nhưng với 10 kỹ thuật sau, em có thể giúp bài làm đạt điểm tối đa trong giới hạn thời gian:

1. **Đọc dữ liệu nhanh:** Dùng sys.stdin.readline thay cho input().

2. **Sử dụng List Comprehension:** [i for i in range(n)] nhanh hơn nhiều so với dùng vòng lặp for và append().
3. **Hàm có sẵn (Built-in):** Luôn ưu tiên sum(), max(), min(), sorted() vì chúng được viết bằng ngôn ngữ C.
4. **Hạn chế biến toàn cục:** Sử dụng biến trong hàm sẽ nhanh hơn biến ngoài hàm.
5. **Nối chuỗi:** Dùng ".join(list) thay vì dùng phép cộng + trong vòng lặp.
6. **Giải nén toán tử:** Dùng * để in mảng nhanh hơn: print(*arr).
7. **Sử dụng Map:** map(int, sys.stdin.read().split()) để xử lý hàng triệu con số trong tích tắc.
8. **Tránh gọi hàm lặp lại:** Thay vì for i in range(len(a)), hãy gán n = len(a) rồi dùng range(n).
9. **Sử dụng math module:** Các hàm như math.gcd(), math.sqrt() được tối ưu hóa cực tốt.
10. **Phép toán Bit:** Dùng <<, >>, &, | thay cho các phép toán số học khi có thể.

Chuyên đề 4: Cách kiểm soát thời gian và bộ nhớ trong phòng thi

Kỳ thi HSG luôn có giới hạn chặt chẽ (Ví dụ: 1.0s và 256MB). Em cần biết cách tự ước lượng:

- **Quy tắc "10 triệu":** Trong , hãy ước lượng máy chấm thực hiện được khoảng **10^7** phép tính mỗi giây. Nếu bài toán có N = 10^5, thuật toán O(N^2) (10 tỷ phép tính) chắc chắn sẽ bị TLE.
- **1. Tại sao lại bị TLE (là viết tắt của Time Limit Exceeded)?**
- Mỗi bài toán đều có một giới hạn thời gian và giới hạn dữ liệu (N). Máy chấm thường xử lý được khoảng **10^8 phép tính mỗi giây**. Nếu thuật toán của bạn có độ phức tạp quá lớn so với N, nó sẽ quá tải.
- Ví dụ thực tế:
- Nếu bài toán cho N = 10^5 và thời gian là 1 giây:
- Thuật toán O(N log N) chạy khoảng 1.7 x 10^6 phép tính -> **Pass (Vượt qua)**.

- Thuật toán O(N^2) chạy khoảng 10^10 phép tính -> **Chắc chắn TLE** (vì gấp 100 lần khả năng của máy).
- **2. Bảng đối chiếu nhanh để tránh TLE**
- Dựa vào giá trị của N trong đề bài, bạn có thể dự đoán thuật toán nào sẽ bị TLE:

Giá trị của N	**Độ phức tạp tối đa cho phép**	**Ví dụ thuật toán**
N <= 10	O(N!)	Hoán vị, quay lui vét cạn
N<= 20	O(2^N)	Duyệt phân tập, quy hoạch động trạng thái
N <= 500	O(N^3)	Nhân ma trận, Floyd-Warshall
N <= 5000	O(N^2)	Sắp xếp chọn, duyệt hai vòng lặp lồng nhau
N <= 10^6	O(N log N) hoặc O(N)	Quick Sort, Tree, Binary Search, Segment Tree
N rất lớn	O(log N) hoặc O(1)	Toán học, tìm kiếm nhị phân trên kết quả

- **3. Cách khắc phục khi bị TLE**
- Nếu bạn nhận thấy mã của mình chắc chắn sẽ bị TLE, hãy thử các hướng sau:
- **Tối ưu độ phức tạp:** Chuyển từ vòng lặp lồng nhau O(N^2) sang dùng Map, Set hoặc kỹ thuật hai con trỏ O(N).
- **Sử dụng cấu trúc dữ liệu mạnh hơn:** Thay vì tìm kiếm tuyến tính, hãy dùng Binary Search hoặc Segment Tree.
- **Tối ưu nhập/xuất:** Trong C++, sử dụng ios_base::sync_with_stdio(0); cin.tie(0); để tăng tốc độ đọc dữ liệu. Tránh dùng endl mà hãy dùng \n.
- **Quy hoạch động:** Nếu bạn đang dùng đệ quy thuần túy và bị tính lại nhiều lần, hãy dùng mảng để lưu kết quả (Memoization).

- **Kiểm soát đệ quy:** có giới hạn độ sâu đệ quy (thường là 1000). Nếu dùng thuật toán đệ quy sâu (như DFS), phải thêm dòng:

```
import sys
sys.setrecursionlimit(200000)
```

- **Tiết kiệm bộ nhớ:** Tránh tạo ra quá nhiều mảng phụ không cần thiết. Với các bài toán ma trận lớn, hãy cẩn thận với lỗi **Memory Limit Exceeded (MLE)**.
- **Chiến thuật Subtask:** Nếu không tìm được thuật toán tối ưu nhất, hãy viết thuật toán "vét cạn" (Brute Force) để lấy trọn điểm ở các Test con có dữ liệu nhỏ.

💡 **Lời nhắn của Thầy Tấn Dân dành cho học sinh:**

"Nắm vững 4 chuyên đề này, các em đã thắng 30% cuộc đua. Sự cẩn thận trong kỹ thuật chính là nền tảng để những ý tưởng thuật toán thăng hoa. Chúc các em luyện tập thật tốt!"

📂 PHẦN 2: SỐ HỌC (LÝ THUYẾT SỐ)

Số học là nền tảng của các bài toán Tin học. Nắm vững phần này giúp các em giải quyết gọn gàng các bài toán về dãy số, tính toán và tối ưu hóa.

Chuyên đề 5: Số nguyên tố - "Chốt chặn" của mọi đề thi

Số nguyên tố là các số chỉ chia hết cho 1 và chính nó. Trong thi HSG, chúng ta cần hai kỹ năng chính:

1. Kiểm tra 1 số N có là số nguyên tố hay không (O(\sqrt{N})):

Thay vì duyệt đến N, chúng ta chỉ cần duyệt đến căn bậc hai của N để tiết kiệm thời gian.

import math

```
def kiem_tra_so_nguyen_to(n):
    if n < 2:
        return False
    # Duyệt từ 2 đến căn bậc hai của n
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True
```

2. Sàng Eratosthenes – Tìm mọi số nguyên tố đến 10^6 (O(N log N)):

Khi đề bài yêu cầu đếm hoặc liệt kê rất nhiều số nguyên tố, kỹ thuật Sàng là bắt buộc.

```
def sang_so_nguyen_to(gioi_han):
    # Khởi tạo danh sách toàn bộ là True (coi là số nguyên tố)
    la_so_nguyen_to = [True] * (gioi_han + 1)
    la_so_nguyen_to[0] = la_so_nguyen_to[1] = False

    for p in range(2, int(math.sqrt(gioi_han)) + 1):
        if la_so_nguyen_to[p]:
            # Đánh dấu các bội của p không phải là số nguyên tố
            for i in range(p * p, gioi_han + 1, p):
                la_so_nguyen_to[i] = False
    return la_so_nguyen_to
```

Chuyên đề 6: Ước chung lớn nhất (UCLN) & Bội chung nhỏ nhất (BCNN)

Đây là công cụ cực mạnh để rút gọn phân số, tìm chu kỳ hoặc giải các bài toán về chia quà, chia lưới.

- **UCLN:** Sử dụng thuật toán Euclid (đã được tích hợp cực nhanh trong thư viện math).
- **BCNN:** Dựa trên công thức: BCNN(a, b) = \frac{|a \times b|}{UCLN(a, b)}.

Code thực chiến:

```
import math

so_a = 12
so_b = 18

ucln = math.gcd(so_a, so_b) # Kết quả: 6
bcnn = (so_a * so_b) // ucln # Kết quả: 36

# Lưu ý: Từ  3.9+, math.gcd có thể tìm UCLN của nhiều số
# ucln_nhieu_so = math.gcd(12, 18, 24, 30)
```

Chuyên đề 7: Phân tích thừa số nguyên tố

Mọi số tự nhiên lớn hơn 1 đều có thể phân tích thành tích các số nguyên tố. Đây là cách để tìm số lượng ước số và tổng các ước số của một số.

Code phân tích thừa số nguyên tố:

```
def phan_tich_thua_so(n):
    uoc = 2
    danh_sach_thua_so = []
    tam = n
    while uoc * uoc <= tam:
        while tam % uoc == 0:
            danh_sach_thua_so.append(uoc)
```

```
            tam //= uoc
        uoc += 1
    if tam > 1:
        danh_sach_thua_so.append(tam)
    return danh_sach_thua_so
```

Chuyên đề 8: Xử lý số siêu lớn & Lũy thừa nhanh

có thể xử lý các con số có hàng nghìn chữ số mà không lo tràn bộ nhớ như C++ hay Pascal.

1. Tính lũy thừa nhanh (Modular Exponentiation):

Khi đề bài yêu cầu tính a^b \pmod m với b cực lớn (ví dụ 10^{18}), chúng ta dùng hàm pow có sẵn của để đạt tốc độ tối đa.

```
# Tính (co_so ^ mu) chia lấy dư cho m
co_so = 2
mu = 10**18
m = 10**9 + 7

ket_qua = pow(co_so, mu, m)
# Chạy cực nhanh so với việc dùng (co_so ** mu) % m
```

2. Xử lý số có hàng nghìn chữ số:

Nếu đề bài cho số cực lớn, hãy đọc dữ liệu dưới dạng xâu (string) rồi mới chuyển sang kiểu số (int).

```
import sys
# Tăng giới hạn chữ số nếu số có hơn 4300 chữ số
sys.set_int_max_str_digits(10000)
```

so_lon = int(input()) # tự động hiểu và tính toán được ngay

💡 **Lời nhắn của Thầy Tấn Dân:**

"Các em hãy chú ý: Trong lập trình, không chỉ cần code đúng mà còn cần code chạy nhanh. Việc hiểu và dùng đúng các hàm như math.gcd hay pow(a, b, m) sẽ giúp các em tiết kiệm thời gian quý báu trong phòng thi để tập trung vào những câu khó hơn!"

Dạ vâng thưa **Thầy Tấn Dân**, đây là nội dung chi tiết cho **PHẦN 3: XỬ LÝ XÂU KÝ TỰ VÀ MẢNG**. Đây là phần kiến thức "xương sống", xuất hiện trong 100% các đề thi Tin học. Em đã Việt hóa toàn bộ tên biến và giải thích cực kỳ cặn kẽ để học sinh của thầy đọc là hiểu ngay ạ.

📂 PHẦN 3: XỬ LÝ XÂU KÝ TỰ VÀ MẢNG

Xâu và Mảng là nơi lưu trữ dữ liệu. Làm chủ được hai cấu trúc này, các em đã nắm trong tay công cụ mạnh nhất để giải quyết mọi bài toán thực tế.

Chuyên đề 9: Kỹ thuật xử lý Xâu ký tự (String)

hỗ trợ xử lý văn bản cực kỳ mạnh mẽ so với C++ hay Pascal. Các em cần nhớ các kỹ thuật "cắt xẻ" sau:

1. Cắt xâu (Slicing): Lấy một phần của xâu mà không cần dùng vòng lặp.

xau = "ThayTanDan"

Lấy từ đầu đến vị trí 3: xau[0:4] -> "Thay"

Lấy xau đảo ngược: xau[::-1] -> "naDnaTyahT"

2. Các hàm bổ trợ "thần thánh":

```
s = "  lap trinh   "
s = s.strip() # Loại bỏ khoảng trắng thừa ở 2 đầu
s = s.upper() # Chuyển thành chữ HOA
s = s.lower() # Chuyển thành chữ thường
dem = s.count("p") # Đếm số lần xuất hiện của chữ "p"
moi = s.replace("", "tin hoc") # Thay thế xâu
```

3. Tách và Nối xâu (Cực kỳ quan trọng để đọc dữ liệu):

```
dong_du_lieu = "10 20 30 40"
danh_sach_so = dong_du_lieu.split() # Tách thành ['10', '20', '30', '40']

danh_sach_chu = ["Hoc", "voi", "Thay", "Dan"]
cau_hoan_chinh = " ".join(danh_sach_chu) # Nối thành "Hoc voi Thay Dan"
```

Chuyên đề 10: Mảng 1 chiều và Mảng 2 chiều (List & Matrix)

Mảng là tập hợp các phần tử. Trong , chúng ta dùng List.

1. Khởi tạo mảng nhanh (List Comprehension):

```
# Tạo mảng chứa 10 số 0
mang = [0] * 10

# Tạo mảng chứa các số bình phương từ 1 đến n
n = 5
binh_phuong = [i*i for i in range(1, n+1)] # [1, 4, 9, 16, 25]
```

2. Mảng 2 chiều (Ma trận): Thường dùng để lưu bản đồ hoặc bảng số.

```
# Khởi tạo ma trận 3 hàng 4 cột toàn số 0
hang, cot = 3, 4
ma_tran = [[0] * cot for _ in range(hang)]

# Truy cập phần tử ở hàng i, cột j: ma_tran[i][j]
```

3. Sắp xếp mảng:

```
mang = [5, 2, 9, 1]
mang.sort() # Sắp xếp tăng dần: [1, 2, 5, 9]
mang.sort(reverse=True) # Sắp xếp giảm dần: [9, 5, 2, 1]
```

Chuyên đề 11: Kỹ thuật Mảng cộng dồn (Prefix Sum)

Đây là kỹ thuật giúp tính tổng một đoạn từ vị trí L đến R trong thời gian siêu nhanh O(1).

- **Bài toán:** Cho mảng A, tính tổng các số từ A[L] đến A[R] (thực hiện Q lần vấn tin).
- **Cách làm:** Tạo mảng tong_tien_to trong đó tong_tien_to[i] lưu tổng từ đầu đến vị trí i.

```
def mang_cong_don(mang_a):
    n = len(mang_a)
    tong_tien_to = [0] * (n + 1)
    for i in range(n):
        tong_tien_to[i+1] = tong_tien_to[i] + mang_a[i]
    return tong_tien_to

# Để tính tổng từ L đến R (chỉ số từ 1):
```

```
# tong_doan = tong_tien_to[R] - tong_tien_to[L-1]
```

Chuyên đề 12: Hai con trỏ (Two Pointers)

Kỹ thuật này dùng để duyệt mảng hiệu quả, thường dùng khi mảng đã được sắp xếp.

Bài toán mẫu: Tìm một cặp số trong mảng đã sắp xếp có tổng bằng K.

```
def tim_cap_so(mang, k):
    trai = 0
    phai = len(mang) - 1

    while trai < phai:
        tong_hien_tai = mang[trai] + mang[phai]
        if tong_hien_tai == k:
            return mang[trai], mang[phai]
        elif tong_hien_tai < k:
            trai += 1 # Cần tổng lớn hơn nên tăng con trỏ trái
        else:
            phai -= 1 # Cần tổng nhỏ hơn nên giảm con trỏ phải
    return None
```

💡 **Lời nhắn của Thầy Tấn Dân:**

"Các em thân mến, Xâu và Mảng giống như gạch và xi măng để xây nên ngôi nhà thuật toán. Nếu các em sử dụng thành thạo các hàm có sẵn như split(), join() hay kỹ thuật Mảng cộng dồn, tốc độ làm bài của các em sẽ nhanh hơn các bạn khác rất nhiều. Hãy luyện tập để đôi tay trở nên thuần thục nhé!"

📂 PHẦN 4: THUẬT TOÁN NÂNG CAO

Đây là những công cụ giúp các em giải quyết các bài toán phức tạp và tối ưu hóa thời gian chạy để đạt điểm tuyệt đối ở những bộ Test khó nhất.

Chuyên đề 13: Đệ quy (Recursion)

Đệ quy là việc một hàm tự gọi lại chính nó. Để không bị lặp vô hạn, mọi bài toán đệ quy phải có **Điểm dừng (Base case)**.

- **Quy tắc 2 phần:**
    1. **Phần dừng:** Khi nào thì không cần gọi lại hàm nữa?
    2. **Phần đệ quy:** Công thức liên hệ giữa bài toán lớn và bài toán nhỏ hơn.

Ví dụ: Tính Giai thừa của N (N!)

```
def tinh_giai_thua(n):
    # Điểm dừng: 0! = 1
    if n == 0:
        return 1
    # Phần đệ quy: n! = n * (n-1)!
    return n * tinh_giai_thua(n - 1)
```

Lưu ý của Thầy Dân: có giới hạn độ sâu đệ quy. Nếu làm bài thi về Đồ thị hoặc Đệ quy sâu, các em nhớ thêm dòng sys.setrecursionlimit(200000) ở đầu bài nhé!

Chuyên đề 14: Thuật toán Tham lam (Greedy)

Tham lam là chiến thuật: **"Tại mỗi bước, ta chọn lựa chọn tốt nhất ở thời điểm hiện tại"** với hy vọng sẽ dẫn đến kết quả tốt nhất toàn cục.

- **Đặc điểm:** Chạy rất nhanh, cài đặt đơn giản nhưng cần chứng minh tính đúng đắn.

- **Bài toán điển hình:** Đổi tiền (với các mệnh giá là bội số của nhau), sắp xếp lịch hội nghị, nối dây thừng.

Ví dụ: Bài toán đổi tiền (Tìm số tờ tiền ít nhất)

```
def doi_tien(so_tien_can_doi, danh_sach_menh_gia):
    # Sắp xếp mệnh giá từ lớn đến nhỏ để tham lam
    danh_sach_menh_gia.sort(reverse=True)
    so_to = 0

    for menh_gia in danh_sach_menh_gia:
        so_to += so_tien_can_doi // menh_gia
        so_tien_can_doi %= menh_gia

    return so_to
```

Chuyên đề 15: Chặt nhị phân (Binary Search)

Đây là thuật toán "quốc dân" để tăng tốc tìm kiếm từ O(N) xuống O(\log N). Điều kiện tiên quyết: **Mảng phải được sắp xếp.**

Code tìm kiếm vị trí của X trong mảng đã sắp xếp:

```
def tim_kiem_nhi_phan(mang, x):
    trai = 0
    phai = len(mang) - 1
    while trai <= phai:
        giua = (trai + phai) // 2
        if mang[giua] == x:
            return giua # Tìm thấy X
```

```
        elif mang[giua] < x:
            trai = giua + 1 # Tìm ở nửa bên phải
        else:
            phai = giua - 1 # Tìm ở nửa bên trái
    return -1 # Không tìm thấy
```

Chuyên đề 16: Quy hoạch động (Dynamic Programming - Cơ bản)

Quy hoạch động là đỉnh cao của tư duy thuật toán. Bản chất là: **Chia bài toán lớn thành các bài toán con và LƯU LẠI kết quả bài toán con để không phải tính lại.**

Ví dụ: Dãy số Fibonacci (Tối ưu bằng cách lưu mảng)

```
def fibonacci_quy_hoach_dong(n):
    # Tạo mảng lưu trữ kết quả các bước trước
    f = [0] * (n + 1)
    f[0], f[1] = 0, 1

    for i in range(2, n + 1):
        f[i] = f[i-1] + f[i-2] # Tính dựa trên kết quả đã có

    return f[n]
```

Bí kíp của Thầy Dân: Nếu em thấy bài toán có thể chia nhỏ và các bài toán con bị lặp lại nhiều lần, hãy nghĩ ngay đến Quy hoạch động!

💡 **Lời nhắn của Thầy Tấn Dân:**

"Các em đừng sợ những cái tên như 'Quy hoạch động' hay 'Chặt nhị phân'. Thực chất chúng chỉ là những cách thông minh hơn để chúng ta dặn máy tính làm việc.

Hãy kiên trì code lại từng ví dụ này, các em sẽ thấy tư duy của mình 'nhảy vọt' chỉ sau vài lần thực hành!"

📂 PHẦN 5: KHO ĐỀ THI & CHIẾN THUẬT "VÉT ĐIỂM"

Kỹ năng giải bài rất quan trọng, nhưng kỹ năng đi thi mới là yếu tố quyết định tấm huy chương. Hãy học cách làm chủ thời gian và tâm lý!

Chuyên đề 17: Cấu trúc đề thi HSG cấp Tỉnh thông thường

Đề thi thường diễn ra trong **150 - 180 phút** với 3 đến 4 câu hỏi có độ khó tăng dần:

1. **Câu 1 (Dễ - 5/20 điểm):** Thường là Số học cơ bản hoặc xử lý Xâu đơn giản. (Mục tiêu: Làm trong 15-20 phút).
2. **Câu 2 (Trung bình - 6/20 điểm):** Xử lý Mảng, Tham lam hoặc Sắp xếp. (Mục tiêu: Làm trong 30-40 phút).
3. **Câu 3 & 4 (Khó - 9/20 điểm):** Quy hoạch động, Đồ thị hoặc Hình học tính toán. Đây là câu phân loại để lấy giải Nhất, Nhì.

Chuyên đề 18: Chiến thuật "Vét điểm" (Subtask Scoring)

Trong Tin học, không phải cứ làm xong cả bài mới có điểm. Đề thi luôn chia làm nhiều **Subtask** (gói điểm nhỏ).

- **Quy tắc "Duyệt trâu" (Brute Force):** Nếu em không nghĩ ra thuật toán tối ưu O(N), hãy đừng bỏ cuộc. Hãy viết một thuật toán đơn giản nhất O(N^2) hoặc O(N^3). Dù không ăn trọn điểm, em vẫn sẽ lấy được 30-50% số điểm của bài đó thay vì 0 điểm.

Ví dụ: Bài toán tìm cặp số có tổng bằng K

- *Cách tối ưu:* Dùng Hai con trỏ hoặc Map (O(N)).
- *Cách "vét điểm" (Duyệt trâu):* Dùng 2 vòng lặp lồng nhau (O(N^2)).

Code "Vét điểm" - Chậm nhưng chắc chắn có điểm với N nhỏ

```
def vet_diem_tong_bang_k(mang, k):
    n = len(mang)
    for i in range(n):
        for j in range(i + 1, n):
            if mang[i] + mang[j] == k:
                return mang[i], mang[j]
    return None
```

Chuyên đề 19: Các lỗi "mất tiền oan" trong phòng thi

Thầy Tấn Dân dặn các em phải kiểm tra kỹ 4 điều này trước khi nộp bài:

1. **Sai tên File:** Chỉ cần sai một dấu chấm, dấu gạch dưới ở tên file .INP hoặc .OUT, bài của em sẽ bị 0 điểm ngay lập tức.
2. **In thừa thông báo:** Máy chấm tự động sẽ so khớp từng ký tự. Nếu em in thêm dòng chữ *"Moi ban nhap n:"* hoặc *"Ket qua la:"*, máy sẽ báo sai đáp án. **Chỉ in đúng thứ đề bài yêu cầu.**
3. **Quên xóa lệnh Debug:** Nhiều em viết lệnh print để kiểm tra lỗi lúc làm bài nhưng quên xóa khi nộp. Điều này dẫn đến sai kết quả.
4. **Kiểu dữ liệu:** tự xử lý số lớn, nhưng với số thực, hãy cẩn thận với số chữ số thập phân (dùng "{:.2f}".format(so) để định dạng).

Chuyên đề 20: Danh mục 10 đề thi trọng điểm (Kho đề thi)

Thầy đã tổng hợp các bộ đề thi từ các tỉnh thành có phong trào Tin học mạnh nhất để các em luyện tập:

- **Đề 1:** Chuyên đề Số học - Đề thi HSG Tỉnh Đồng Nai 2024.
- **Đề 2:** Chuyên đề Xâu ký tự - Đề thi HSG TP. Hồ Chí Minh 2023.
- **Đề 3:** Chuyên đề Mảng & Sắp xếp - Đề thi Olympic 30/4.
- **Đề 4:** Chuyên đề Quy hoạch động - Đề thi thử vào Chuyên Tin.
- *(Và tiếp tục 6 đề thực chiến khác kèm lời giải chi tiết...)*

💡 **Lời nhắn cuối cùng của Thầy Tấn Dân:**

"Các trò yêu quý, bước vào phòng thi, điều quan trọng nhất không phải là em biết bao nhiêu thuật toán, mà là em giữ được **cái đầu lạnh**. Hãy làm câu dễ thật cẩn thận để không mất điểm, sau đó bình tĩnh 'vét' từng điểm ở câu khó. Thầy tin tấm bằng giải Tỉnh đang chờ đợi những người kiên trì và khôn ngoan nhất. Chúc các em tự tin và chiến thắng!"

HÀNH TRÌNH VẠN DẶM BẮT ĐẦU TỪ MỘT DÒNG CODE!

Cảm ơn các em đã kiên trì theo dõi hết bộ bí kíp này. Thành công trong kỳ thi HSG không chỉ dành cho người thông minh nhất, mà dành cho người chuẩn bị kỹ càng nhất.

Lớp bồi dưỡng Tin học Thầy Tấn Dân vẫn đang tuyển sinh:

- **Hình thức:** Học Online/Trực tiếp.
- **Đối tượng:** Học sinh lớp 6,7,8,9 ôn thi cấp Tỉnh, vào chuyên Tin.
- **Cam kết:** Lộ trình bài bản - Tặng kho đề thi độc quyền.

👉 **Đăng ký ngay tại Zalo:** 0937 179 278 (Thầy Tấn Dân)